Student Marks Management API

Assignment 1 | Advanced Programming (C# and .NET)

Architecture

The solution is an ASP.NET Core 8 Web API backed by PostgreSQL through Entity Framework Core. Controllers handle HTTP and authorization, service interfaces implement business rules, DTOs define request/response contracts, and AppDbContext configures the relational model. EF Core migrations create and evolve the database. Swagger documents the API.

Data model

Students and Courses are linked through Enrollments, which carries an optional mark. Users store password hashes and a role; student accounts link to their student record. Unique indexes, foreign keys, delete restrictions and check constraints keep the database consistent. The ER diagram is included as erd.png.

Security and business rules

Authentication and authorization

Passwords are verified using ASP.NET Core PasswordHasher and users receive short-lived JWTs. Teacher-only endpoints enforce the Teacher role. Student record and transcript endpoints compare the authenticated student_id claim with the requested ID. Connection strings and JWT signing keys are configured through user-secrets or environment variables, not committed appsettings values.

Validation and errors

Request DTO validation rejects missing, oversized or out-of-range values at the API boundary. Errors use Problem Details. Database unique violations are translated into HTTP 409 Conflict; check violations become 400 Bad Request. Duplicate records use 409 because they conflict with the current database state rather than being malformed requests.

Concurrency and reports

Unique constraints are the final guarantee against concurrent duplicate registration numbers, emails, course codes and duplicate enrollments; the API translates database conflicts instead of relying on a check-then-insert sequence. Course statistics use graded marks only. Transcript averages are weighted by credit hours across graded courses.

Running and verification

Running the project

Create the PostgreSQL database and user as described in README.md, configure ConnectionStrings:Database and Jwt:Key with dotnet user-secrets, then run dotnet ef database update and dotnet run. Demo credentials are development-only and documented in README.md.

Testing

The Postman v2.1 collection covers the API and negative cases. Import it with the Local environment and run against a fresh database. The opt-in smoke module is started with dotnet run -- --smoke-test; it uses HTTP requests against the real API and database, prints expected/actual status codes and returns a non-zero exit code on failed checks.

Submission note

Postman Runner and smoke-test screenshots must be captured after running against a configured PostgreSQL instance. Replace the repository link on the report cover page with the private repository URL before submission. The author should review the code and be prepared to explain design decisions during the viva.